

Master Thesis “The Game Beneath the Game:
The Evolution of Cheats and Anti-Cheat
Systems in Counter-Strike 2”

Pablo Valiente

Tutor: Antonio Nappa
Universidad Carlos III de Madrid
Date: February 2025

Abstract

This thesis examines the evolution of cheating in video games, from early debugging tools to modern external and internal exploits. It explores key cheating techniques, anti-cheat countermeasures, and the ongoing arms race between both sides. Additionally, a practical implementation of a cheat in Counter-Strike 2 is developed to assess the effectiveness of contemporary anti-cheat systems and identify potential vulnerabilities.

Keywords: Cheating, Video Games, Counter-Strike 2, Anti-Cheat Systems, Game Hacking, Memory Manipulation

DEDICATION

CONTENTS

1	Introduction	2
1	Context and Motivation	2
2	Objectives and Scope	3
3	Competencies and Alignment with the Master's Program	3
4	Document Structure	5
2	From Primitive Survival to Digital Domination	6
1	The Competitive Nature of Humanity	6
2	The Birth of Computation and the Internet: A New Digital Battle-ground	7
2.1	Early Computational Tools: From the Abacus to Mechanical Calculators	7
2.2	The 19th-Century Revolution: The Birth of Programmable Machines	8
2.3	The Acceleration of Computation in the 20th Century	8
2.3.1	World War II and the Role of Computation	8
2.3.2	Post-War Expansion: From Military to Commercial Computing	9
2.4	The Birth of the Internet: From Military Networks to Global Infrastructure	9
2.5	The Digital Arena: From Computing to Competition	10
2.6	From Networks to Virtual Battlefields	10
3	The Evolution of Cheating: From Debugging Tools to an Organized Industry	12
1	The Early Days: Debugging Tools and Experimental Modifications	12
2	The Rise of Cheating in Consoles and PC	12
2.1	Cheat Devices: Game Genie, GameShark, and Pro Action Replay	13
3	The Transition to Online Gaming: Cheating Becomes a Competitive Threat	14
4	The Birth and Evolution of Anti-Cheat Systems	14
5	The Commercialization of Cheating	16
4	Injection and Detection Techniques	19
1	Understanding Manual Mapping	19
2	TLS Callbacks as a Detection Primitive	20

3	Evasion and Limitations	21
4	VAC-Specific Capabilities	23
5	Summary	23
5	Internal Game Structure: Logic Behind Counter-Strike 2	24
1	Understanding Entity Lists in Video Games	24
1.1	Common Structures for Entity Lists	24
2	Entity Organization in Counter-Strike 2	25
2.1	Structuring Player Data in Practice: The Players Class	27
3	Entity List Traversal	28
3.1	Entity Handle Relationships	28
3.2	Locating and Identifying the Entity List in Memory	28
3.3	Reverse Engineering the Engine's Entity Resolution Logic	30
3.3.1	Alternative Method: Hooking OnAddEntity and On- RemoveEntity	31
6	Cheat Features Implemented in Our Cheat for CS2	33
1	Chams: Enhancing Visibility Through Engine-Based Rendering	34
1.1	Extracting Default Materials from the Game Engine	35
1.2	Generating Custom Materials	36
1.3	Hooking CreateMaterial and Injecting Custom Materials	38
1.4	Loading KV3 Materials into the Game	38
1.5	Hooking OnDrawModelExecute and Overriding Materials	39
1.5.1	Filtering Target Entities	40
1.5.2	Overriding Material Pointers	40
1.5.3	Visible vs Non-Visible Chams Logic	41
2	Aimbot: Automated Aim to Enemy Players	42
2.1	Entity Management: Hooking the Game's Entity System	42
2.1.1	Computing Aiming Angles	44
2.1.2	Deriving the Direction Vector	44
2.1.3	Computing the Player's Position	45
2.1.4	Centering Coordinates on the Enemy	46
2.1.5	Local Coordinate System and Angle Calculation	46
2.1.6	Mathematical Computation of Aiming Angles	48
2.2	Overwriting Player View Angles	48
2.3	Smoothing for Stealth	49
3	Bone Calculation: Extracting and Rendering Player Skeletons	50
3.1	Extracting Bone Data from Player Entities	50
3.2	Visualizing Bone Indices on the Player Model	51
3.3	Rendering a Connected Skeleton	52
	Bibliography	54

LIST OF FIGURES

3.1	Game Genie	13
6.1	Created Chams Materials	41
6.2	Enter Caption	42
6.3	Representation of Pitch, Yaw, and Roll in a 3D Space.	45
6.4	Representation of the player and enemy positions on the map.	46
6.5	Computation of the displacement vector D from the player's eye to the target.	47
6.6	3D representation of the aiming direction vector.	47
6.7	Visualizing all bone indices rendered over the player model.	51
6.8	Simplified player skeleton rendered using bone connections.	53

LIST OF TABLES

6.1	Core Functionalities Overview	33
-----	---	----

1. INTRODUCTION

1 Context and Motivation

The evolution of video games from a niche entertainment medium to a dominant digital industry has led to the rise of competitive gaming, where players engage in skill-based challenges to outperform their opponents. The widespread availability of high-speed internet and global connectivity has further transformed online multiplayer games into structured competitive ecosystems, characterized by rankings, tournaments, and financial incentives that continuously drive players to refine their strategies and skills.

Today, video games are an integral part of millions of lives worldwide, serving not only as entertainment but also as social platforms, economic engines, and professional career paths. However, the increasing competitiveness and monetization of gaming have also fueled the prevalence of cheating, where players exploit unauthorized modifications to gain an unfair advantage. These methods range from memory manipulation and external assistance tools to network-based exploits and hardware modifications. The ongoing arms race between game developers—who continuously strengthen anti-cheat mechanisms—and cheat developers—who evolve their techniques to bypass these defenses—has made cheating one of the most pressing challenges in modern gaming.

At the same time, the implementation of anti-cheat systems introduces significant privacy, legal, and environmental dilemmas. Advanced detection methods often require deep system access, raising concerns over user privacy and digital rights. Meanwhile, the large-scale commercialization of cheats results in substantial financial losses for game developers and complex regulatory challenges. Additionally, the computational cost of sophisticated anti-cheat solutions contributes to increased energy consumption, raising further environmental concerns.

To go beyond theoretical discussions and ethical concerns, this research will delve into the technical intricacies of cheating by implementing and analyzing a custom-built cheat within a large-scale multiplayer game. This approach will allow for an in-depth evaluation of modern anti-cheat systems, providing empirical insights into their strengths, weaknesses, and potential bypasses.

By combining historical analysis, technical research, and practical implementation, this study aims to offer a comprehensive understanding of cheating in competitive gaming. A case study in a large-scale multiplayer game will serve as the foundation

for assessing the effectiveness of contemporary anti-cheat measures and their broader societal impact.

2 Objectives and Scope

This study aims to analyze cheating in competitive video games from both a theoretical and practical perspective. It will explore its historical evolution, technical mechanisms, and countermeasures, while also assessing the effectiveness of modern anti-cheat systems through hands-on experimentation.

To achieve this, the research will focus on three key areas:

1. The evolution of cheating in competitive gaming, examining its impact on game design, player behavior, and industry responses.
2. A technical analysis of prevalent cheating techniques, including aimbots, wall-hacks, network manipulation, and memory injection, alongside their implications for competitive integrity.
3. The implementation and evaluation of a custom cheat in a large-scale multiplayer game, providing empirical insights into the strengths and weaknesses of current anti-cheat solutions.

3 Competencies and Alignment with the Master's Program

This thesis aligns with the core competencies of the Master's Degree in Cybersecurity, equipping professionals with essential skills in reverse engineering, intrusion analysis, and digital forensic investigation. The key competencies developed in this study are:

Basic Competencies

- [CB6] Conducting original research on competitive gaming security, analyzing cheating techniques and countermeasures.
- [CB7] Applying cybersecurity principles to game security, focusing on cheat development, detection, and evasion.
- [CB8] Evaluating the ethical and legal considerations of game manipulation, with implications for software security and fair competition.

General Competencies

- [CG1] Investigating and replicating game manipulation techniques to assess vulnerabilities in competitive gaming environments.
- [CG2] Examining the role of anti-cheat systems in detecting unauthorized modifications and analyzing their effectiveness.

- [CG3] Producing well-documented research on offensive security methodologies, including memory hacking and system intrusion.

Specific Competencies

- [CE1] Analyzing the psychological motivations behind cheating in competitive gaming.
- [CE2] Investigating modern cheat evasion techniques and their ability to bypass detection.
- [CE3] Researching trends in game security threats and drawing parallels to broader cybersecurity challenges.
- [CE4] Conducting forensic analysis of game environments to extract digital evidence of unauthorized modifications.
- [CE6] Assessing the architecture and performance of modern anti-cheat systems, evaluating their strengths and weaknesses.

4 Document Structure

This thesis is structured as follows:

- Chapter 1: Introduction – Provides the research context, outlining the motivations, objectives, and competencies addressed in the study, along with an overview of the document’s structure.
 - Chapter 2: From Primitive Survival to Digital Domination – Explores the competitive nature of humanity and the evolution of computation, from early mechanical tools to the internet era, establishing the digital landscape in which competitive gaming emerged.
 - Chapter 3: The Evolution of Cheating: From Debugging Tools to an Organized Industry – Analyzes the historical development of cheating in video games, tracing its origins from early debugging tools to modern exploit markets and cybercriminal networks.
 - Bibliography – Compiles the academic and technical references used throughout the research.
-

2. FROM PRIMITIVE SURVIVAL TO DIGITAL DOMINATION

1 The Competitive Nature of Humanity

Throughout history, humanity has been shaped by an innate drive for superiority, dominance, and recognition. From primitive survival instincts to modern technological achievements, the desire to outperform others has fueled progress in every field—science, warfare, politics, sports, and, in more recent times, digital environments.

This competitive nature is not merely a byproduct of ambition but a deeply ingrained psychological mechanism that has guided human behavior for centuries. Understanding the psychological foundations of this drive is crucial to explaining its persistence and evolution over time. Festinger (1954), through his Social Comparison Theory, posited that individuals have an inherent tendency to evaluate themselves in relation to others. This constant self-assessment fosters competition as people seek to validate their abilities and secure a sense of superiority. Furthermore, McClelland (1961), in his Achievement Motivation Theory, demonstrated that the drive to set goals and surpass one's peers is not simply a social construct but an intrinsic component of human motivation, crucial for both personal fulfillment and status recognition. These psychological insights reveal why competition remains a defining force in human interactions, shaping not only individual aspirations but also broader societal structures.

From an evolutionary perspective, competition has been a fundamental survival mechanism, shaping the trajectory of human development. The earliest human societies relied on strategic thinking and adaptability to outcompete rivals in hunting, warfare, and resource acquisition. As civilization advanced, this drive extended beyond mere survival, influencing cultural, intellectual, and economic structures. Even from childhood, individuals naturally seek to establish their competence relative to others, reinforcing competition as a deeply rooted cognitive and social process.

Thus, what began as a necessity for survival has evolved into a complex, multifaceted force that continues to define human ambition and progress.

With the rise of modern technology, digital environments have emerged as a new arena for competition. The invention of computers, followed by the creation of the internet, revolutionized how humans interact, process information, and engage in

competitive activities. The transition from early computational machines to networked systems paved the way for video games, where structured digital challenges allow individuals to test their abilities against both artificial intelligence and human opponents. These digital platforms tap into the same fundamental competitive instincts seen in traditional settings, providing an environment where players can measure their skills, engage in goal-oriented behavior, and gain recognition within their respective communities. This further reinforces the notion that competition, regardless of its medium, remains a powerful driver of human engagement and achievement.

As in other competitive domains, players in video games seek to optimize their performance, discover hidden mechanics, and exploit advantageous strategies. Research on competitive behavior suggests that when individuals encounter barriers to success, they often look for alternative ways to improve their standing (Murdock et al., 2001). This phenomenon is particularly pronounced in online multiplayer games, where leaderboards, ranking systems, and social validation create high levels of pressure to excel. Whether through pure skill, strategic knowledge, or leveraging unintended mechanics, players are constantly seeking an edge over their opponents.

To fully understand how digital competition has evolved into modern gaming culture, it is essential to examine the historical progression that led to this moment. The following chapters will explore the origins of computation, the development of networked systems, and the emergence of video games as a dominant form of entertainment. By tracing this technological evolution, we will gain insight into how human competition has been redefined in the digital age, leading to the creation of virtual battlefields where skill, innovation, and adaptability determine success.

2 The Birth of Computation and the Internet: A New Digital Battleground

The journey toward digital competition began with humanity's earliest attempts to automate reasoning and computation. Since antiquity, civilizations have sought ways to enhance their capacity for mathematical problem-solving, recognizing its crucial role in commerce, astronomy, and engineering. As societies advanced, so did their tools, gradually evolving from rudimentary counting mechanisms to sophisticated machines capable of complex calculations.

2.1 Early Computational Tools: From the Abacus to Mechanical Calculators

To facilitate numerical tasks, ancient civilizations developed devices that enhanced computational accuracy and efficiency. Among the earliest was the Abacus (circa

2400 BCE), widely used across Mesopotamia, China, and later Europe. By allowing merchants and scholars to perform arithmetic operations swiftly, this tool laid the foundation for mechanical computation.

By the 17th century, advancements in mechanical engineering led to more sophisticated devices, such as Pascal's Adding Machine (1642). Invented by Blaise Pascal, this early mechanical calculator used a system of rotating gears to perform basic arithmetic operations, marking a significant step toward reducing cognitive labor in mathematical tasks. Such innovations foreshadowed the emergence of machines that could not only compute but also execute logical operations.

2.2 The 19th-Century Revolution: The Birth of Programmable Machines

Despite these early developments, computation remained a largely manual process until the 19th century, when the concept of programmable machines emerged. The most influential figure in this transition was Charles Babbage, who designed the Analytical Engine—a theoretical device that introduced fundamental computing principles such as conditional logic, input/output processing, and memory storage. Although never constructed in his lifetime, this innovation established the conceptual groundwork for modern computing.

Building upon Babbage's vision, Ada Lovelace recognized that such machines could transcend mere arithmetic calculations. Her development of the first algorithm for the Analytical Engine positioned her as the world's first programmer. More importantly, her insights into computational logic and symbolic processing laid the foundation for modern programming languages, demonstrating that machines could be programmed to execute intricate sequences of instructions.

2.3 The Acceleration of Computation in the 20th Century

The 20th century marked a turning point in computational development, driven by both military necessity and scientific ambition. The transition from mechanical calculators to fully electronic computers revolutionized data processing, transforming computation from a supportive tool into an essential driver of technological progress.

2.3.1 World War II and the Role of Computation

One of the most significant catalysts for computing advancements was World War II, where rapid and precise calculations were required for cryptographic warfare, ballistic trajectory analysis, and logistical coordination. During this period, Alan Turing formalized the concept of algorithmic processing through his Turing Machine, providing the theoretical model for modern computers. His contributions to

cryptanalysis, particularly in breaking the Enigma cipher, underscored the strategic importance of computational power.

Parallel to Turing's work, large-scale electronic computers were developed to support wartime efforts. The British-built Colossus (1943), one of the first programmable electronic computers, was instrumental in deciphering enemy communications. Meanwhile, in the United States, the creation of ENIAC (1945) enabled rapid ballistic calculations, enhancing military planning and precision. These advancements established computation as a powerful tool for automated problem-solving, demonstrating its potential beyond manual calculations.

2.3.2 Post-War Expansion: From Military to Commercial Computing

Following the war, computing technology expanded into academia, industry, and eventually, everyday life. The late 1940s and early 1950s saw the rise of mainframes, large-scale machines used by governments and research institutions for data processing, scientific simulations, and business applications. This period also witnessed the development of the transistor (1947), which replaced vacuum tubes, making computers smaller, more efficient, and commercially viable.

The transition toward personal computing accelerated with the advent of integrated circuits in the 1960s, leading to the emergence of personal computers (PCs) in the 1970s and 1980s. Companies such as IBM, Apple, and Microsoft played a pivotal role in bringing computing to homes and workplaces. The introduction of graphical user interfaces (GUIs), exemplified by the Apple Macintosh (1984), transformed computers from specialized industrial tools into everyday consumer devices.

2.4 The Birth of the Internet: From Military Networks to Global Infrastructure

While computing power had grown exponentially, its true transformation came with the advent of networked computing. The first major step in this direction was the creation of ARPANET in the late 1960s, a project funded by the U.S. Department of Defense. Designed as a resilient, decentralized communication network, ARPANET introduced packet-switching, a method of transmitting data in independent packets that improved reliability and efficiency.

In the early 1980s, the standardization of TCP/IP protocols allowed different networks to communicate seamlessly, marking the birth of the modern Internet. The introduction of the Domain Name System (DNS) in 1984 simplified navigation, replacing numerical IP addresses with human-readable domain names. However, the most transformative moment came in 1991 with the development of the World Wide Web by Tim Berners-Lee, which enabled hyperlinked browsing and revolutionized digital interaction.

By the mid-1990s, the internet had transitioned from a military and academic tool to a global infrastructure, fueled by the proliferation of personal computers and the expansion of commercial internet service providers (ISPs). This transformation redefined human interaction, enabling not only global communication but also the rise of competitive digital spaces.

2.5 The Digital Arena: From Computing to Competition

As computing evolved into an interconnected ecosystem, competition within digital environments intensified. No longer confined to physical constraints, individuals and organizations sought new ways to optimize systems, manipulate algorithms, and dominate digital spaces. Hackers, programmers, and early adopters of networked technology engaged in skill-based challenges, exploring vulnerabilities and testing the limits of online security.

Simultaneously, the rise of online connectivity redefined one of computing's most engaging applications: gaming. Once a solitary activity or limited to local multiplayer settings, video games evolved into a global competitive platform where players could test their skills against opponents from around the world. The introduction of online multiplayer capabilities in the late 1990s and early 2000s transformed digital gaming from an entertainment medium into an arena for structured competition.

The shift from casual gaming to competitive online play was driven by several technological advancements. Faster internet connections, particularly the spread of broadband, enabled real-time multiplayer interactions with minimal latency. Matchmaking systems allowed players to be paired based on skill level, ensuring fairer and more engaging contests. Leaderboards and ranking systems fostered a sense of progression and status, reinforcing the competitive drive among players. As a result, gaming ceased to be just a recreational activity—it became a structured and measurable form of digital competition.

2.6 From Networks to Virtual Battlefields

The increasing complexity of networked systems created digital battlegrounds where individuals and teams competed for dominance. Online gaming communities flourished, with players dedicating time to mastering mechanics, developing strategies, and optimizing their performance. Titles such as Counter-Strike, StarCraft, and Quake set early standards for competitive gaming, emphasizing skill, reflexes, and tactical awareness.

Unlike other digital challenges, video games provided transparent rules, structured rankings, and quantifiable performance metrics. These elements made gaming one of the most accessible forms of competitive engagement, allowing players worldwide to test their abilities on a level playing field. The introduction of dedicated esports

tournaments and professional leagues further legitimized gaming as a form of structured competition, attracting sponsorships, media attention, and substantial prize pools.

The rise of competitive gaming was not merely an evolution of digital entertainment—it was a natural extension of human competition into the virtual realm. What began as casual online matches evolved into organized tournaments, professional leagues, and eSports, where players compete for prestige, financial rewards, and international recognition.

As technology continues to redefine the nature of digital interaction, gaming stands as one of the most structured and influential arenas of online competition. The following chapters will explore the evolution of competitive gaming, tracing its origins from early arcade tournaments to its establishment as a global entertainment and professional industry.

3. THE EVOLUTION OF CHEATING: FROM DEBUGGING TOOLS TO AN ORGANIZED INDUSTRY

The history of cheating in video games is deeply intertwined with the evolution of the industry itself. What began as debugging tools for developers gradually transformed into a clandestine culture of modifications, and with the advent of online gaming, an organized and profitable industry with ties to cybercrime (Consalvo, 2007). Simultaneously, the fight against cheating led to the development of anti-cheat systems, which evolved from simple integrity checks to sophisticated solutions incorporating artificial intelligence and deep system access (Yan & Randell, 2005).

1 The Early Days: Debugging Tools and Experimental Modifications

Before video games became a mainstream form of entertainment, early cheating emerged as a necessity within software development. In the 1970s and 1980s, developers embedded debugging tools within their games to facilitate testing. These systems allowed programmers to modify internal values such as lives, ammunition, and speed, enabling them to test different scenarios without playing through the entire game (Smith, 2018).

Some of these tools remained accessible in commercial releases, either unintentionally or as a deliberate choice by developers. This led to the creation of the first cheat codes, which were key combinations or commands that altered gameplay. One of the most iconic examples is the Konami Code ($\uparrow \uparrow \downarrow \downarrow \leftarrow \rightarrow \leftarrow \rightarrow$ B A), introduced in *Gradius* (1986) and later popularized in *Contra* (1988) (Kent, 2001). While these codes were not designed to affect competition, they set a precedent for the intentional modification of gameplay.

2 The Rise of Cheating in Consoles and PC

As the video game industry grew throughout the 1980s and 1990s, cheating expanded beyond debugging tools and built-in cheat codes. With the advent of home consoles, external devices specifically designed to modify gameplay began to appear.

2.1 Cheat Devices: Game Genie, GameShark, and Pro Action Replay

The first dedicated cheat devices were modification cartridges, such as the Game Genie (1990) and the GameShark (1995). These hardware add-ons allowed players to inject custom modifications into a game's memory, altering key parameters such as health, ammunition, and physics (Burnham, 2001). While initially marketed as harmless tools for enhancing single-player experiences, they introduced the concept of real-time game manipulation, which would later be adapted for online environments.

On the PC side, the introduction of memory editors provided users with a more flexible and powerful way to modify game values at runtime. Early programs like Cheat Engine (2000) enabled players to search for specific in-game variables and alter them directly, bypassing the limitations of built-in cheat codes (Ramsay, 2012). This shift from developer-sanctioned cheats to unauthorized external modifications laid the foundation for more advanced game-hacking techniques.

As gaming technology evolved, so did the tools available for modifying game memory structures. More sophisticated programs such as ReClass.NET emerged, allowing users to analyze and modify game memory at a structural level. Unlike traditional memory scanners, ReClass.NET enables users to reverse-engineer game memory layouts by dynamically inspecting class structures and data offsets in real-time (Wang, 2015). This advancement significantly improved the ability of cheat developers to create complex modifications, from automatic aimbots to more intricate exploits that manipulate how game logic processes data.



Figure 3.1: Game Genie

These tools, originally designed for educational and debugging purposes, soon became widely used in cheat development, particularly in online gaming environments. Their increasing complexity meant that anti-cheat systems had to adapt rapidly, leading to an ongoing technological arms race between cheat developers and security teams.

3 The Transition to Online Gaming: Cheating Becomes a Competitive Threat

The rise of online multiplayer gaming in the late 1990s and early 2000s fundamentally changed the impact of cheating. Unlike single-player cheats, which only affected the individual experience, online cheating directly disrupted fair competition (Yan & Randell, 2005). As games like *Quake* (1996), *Counter-Strike* (1999), and *Diablo II* (2000) introduced matchmaking and ranking systems, the incentive to cheat increased dramatically (Young, 2016).

The growing importance of competitive integrity led developers to take more aggressive measures against cheating, resulting in the creation of server-side and client-side anti-cheat mechanisms. However, as anti-cheat solutions became more sophisticated, so too did cheating techniques, leading to an ongoing technological arms race between game security teams and cheat developers.

This shift led to the development of the first organized cheat communities, where players and programmers shared exploits, scripts, and modified game clients. It also marked the beginning of an arms race between cheat developers and game security teams, as new forms of cheating emerged alongside increasingly complex anti-cheat measures.

4 The Birth and Evolution of Anti-Cheat Systems

As online gaming became increasingly competitive, the integrity of fair play was challenged by the widespread use of cheats. Unlike single-player modifications, which had no broader consequences, cheating in multiplayer environments compromised matchmaking, rankings, and esports competitions, creating frustration among players and damaging the credibility of competitive gaming.

Early attempts to mitigate cheating relied on server-side validation, where game servers monitored player actions for anomalies that suggested external manipulation. These checks aimed to detect unnatural behavior, such as impossible movement speeds, improbable accuracy, and modifications to game assets that altered visibility constraints. However, this approach had significant limitations, as many cheats operated entirely on the client side, modifying game data before it was transmitted to the server. As a result, more sophisticated client-side anti-cheat solutions were

developed to monitor and detect unauthorized modifications directly within the player's system.

One of the first dedicated anti-cheat programs was PunkBuster, developed by Even Balance, Inc. and introduced in 2000. Initially implemented in *Return to Castle Wolfenstein* and later in *Battlefield 1942*, PunkBuster marked a shift from basic server-side detection to active client-side monitoring. It worked by scanning a player's system for known cheat signatures, verifying game file integrity, and capturing periodic in-game screenshots to identify visual modifications such as wallhacks. Additionally, PunkBuster maintained a global ban list, preventing known cheaters from accessing protected servers. Despite these advancements, cheat developers quickly adapted by employing techniques such as code obfuscation, allowing their software to remain undetected. PunkBuster also faced criticism for its aggressive scanning approach, which occasionally resulted in false positives, banning legitimate players due to conflicts with background applications.

In 2002, Valve introduced Valve Anti-Cheat (VAC) alongside *Counter-Strike 1.6*, implementing a more automated and data-driven approach to cheat detection. Unlike PunkBuster, which immediately expelled suspected cheaters, VAC introduced a delayed banning system. This strategy allowed suspected cheaters to continue playing while logging their infractions, making it harder for cheat developers to identify detection triggers and modify their cheats accordingly. In addition to signature-based detection, VAC utilized heuristic analysis to recognize abnormal behavioral patterns, refining its detection algorithms over time. By integrating server-side validation with machine learning techniques, VAC continuously improved its ability to detect and counteract evolving cheats. However, since it operated at the user level rather than the kernel level, advanced cheats that manipulated system memory with higher privileges could still bypass its protections.

As cheat developers refined their evasion techniques, more aggressive anti-cheat solutions emerged. One of the most notable advancements was BattleEye, introduced in 2004 and later adopted in games like *Arma 2*, *Rainbow Six Siege*, and *PUBG*. Unlike VAC, which prioritized passive detection, BattleEye enforced stricter security measures by running with elevated system privileges. By operating at the kernel level, it could detect and block cheat software that manipulated game memory at a deeper level, making it significantly harder for external programs to alter in-game data undetected. This approach allowed BattleEye to intercept and prevent unauthorized modifications before they could affect gameplay, offering a more proactive defense against sophisticated cheats.

As competitive gaming and esports continued to grow, Riot Games introduced a new anti-cheat solution, Vanguard, designed specifically for *Valorant*. Released in 2020, Vanguard took anti-cheat enforcement to an unprecedented level by implementing persistent kernel-level monitoring, meaning it actively ran as a background process

from the moment the operating system started. This deep system integration enabled it to detect and prevent unauthorized modifications at the lowest levels of system memory, making it one of the most invasive yet effective anti-cheat solutions. However, its intrusive nature sparked controversy among players concerned about privacy, as the software had continuous access to system processes even when the game was not running.

The evolution of anti-cheat systems reflects an ongoing technological arms race between cheat developers and game security teams. While early efforts relied on rudimentary server-side validation, modern solutions integrate behavioral analysis, machine learning, and kernel-level access to provide more robust protection. Despite these advancements, the constant adaptation of cheat developers ensures that no anti-cheat system remains impenetrable for long, perpetuating a continuous cycle of exploitation and countermeasures in the digital battlefield of competitive gaming.

5 The Commercialization of Cheating

The rapid commercialization of cheating has transformed it from a niche activity into a highly organized industry, offering both software-based and hardware-level exploits. Specialized forums such as UnKnoWnCheaTs, MPGH, and ElitePVPers have become central hubs for distributing cheats, providing resources for reverse engineering, memory manipulation, and anti-cheat evasion. These platforms, along with encrypted messaging services like Discord and Telegram, have facilitated the sale and distribution of sophisticated cheats through subscription models and private communities.

As cheat developers adopt increasingly advanced techniques, such as kernel-level drivers and DMA-based exploits, anti-cheat systems are forced into a continuous state of adaptation. The emergence of software-based cheats, particularly those using DLL injection, code hooking, and API interception, has proven to be one of the most adaptable and challenging threats. These methods allow cheat developers to modify game functions dynamically, bypassing traditional detection systems.

From History to Implementation: A Technical Transition

Having traced the historical and sociotechnical evolution of cheating—from its roots in debugging tools to its current status as a commercialized, clandestine industry—we now shift focus toward the technical foundations that sustain it today.

The next chapters will abandon the literary and historical perspective in favor of a rigorous, low-level examination of how modern software-based cheats are constructed and deployed. We will explore the inner mechanics of code injection, memory manipulation, and anti-cheat evasion techniques, providing a detailed view of

the technological arms race from a reverse engineering and systems programming standpoint.

With the historical context established, it is time to dissect the machinery.

4. INJECTION AND DETECTION TECHNIQUES

Modern cheat development relies heavily on code injection—techniques that introduce arbitrary code into a game process to manipulate behavior, render information, or automate gameplay. With increasingly sophisticated anti-cheat systems like Valve Anti-Cheat (VAC), traditional injection methods such as `LoadLibrary` or `CreateRemoteThread` have become obsolete due to their predictability and ease of detection. As a result, cheat developers have adopted more advanced methods like Manual Mapping, which currently represents the gold standard in stealth and effectiveness.

Manual Mapping is widely recognized as the most effective and stealthy method for injecting unauthorized code into game processes like Counter-Strike 2. Unlike traditional injection methods that rely on well-known Windows APIs, Manual Mapping simulates the PE loader's behavior manually. It avoids triggering typical detection vectors monitored by VAC, such as API calls, DLL registration in the PEB, and new thread creation.

1 Understanding Manual Mapping

Manual Mapping replicates the Windows PE loader's workflow entirely in user space. The injector reads the PE headers of a DLL, allocates remote memory, writes the necessary sections, manually resolves imports, performs relocations, and executes the entry point—often through shellcode or hijacked threads—without ever calling `extttLoadLibrary` or registering the DLL with the operating system.

Why Manual Mapping is Superior for CS2 Cheats

In the context of VAC, Manual Mapping stands out for several reasons:

- It avoids detection from module enumeration since the injected DLL never appears in the target's PEB.
- It bypasses DLL injection flags and avoids triggering monitored APIs like `extttCreateRemoteThread` or `extttLoadLibrary`.
- Execution can be achieved through stealth techniques like thread hijacking or return address spoofing, masking malicious control flow.

- It enables further obfuscation, such as encryption of payloads or restoring original memory after patching.

Manual Mapping Steps

1. Parse the PE headers of the DLL.
2. Allocate memory in the target process using VirtualAllocEx.
3. Write the DLL's sections into the allocated memory.
4. Manually resolve imports using GetProcAddress.
5. Perform base relocations to correct address offsets.
6. Transfer execution using thread hijacking or a trampoline.

2 TLS Callbacks as a Detection Primitive

One of the most effective primitives available to anti-cheat developers is the use of Thread Local Storage (TLS) callbacks. Defined within the PE format, these callbacks are executed automatically by the Windows loader whenever a thread is created or a module is loaded. This occurs before the thread reaches its intended start address, providing a unique window for inspection.

Anti-cheats use this to examine newly created threads before they begin execution. A typical implementation involves querying the thread's start address using NtQueryInformationThread, then verifying whether this address lies within known, legitimate modules or in anonymous memory.

Simplified TLS Detection Code for Cheats

The following example illustrates how an anti-cheat might use a TLS callback to detect potentially injected threads:

```

1 void NTAPI tls_callback(PVOID, DWORD reason, PVOID) {
2     if (reason != DLL_THREAD_ATTACH && reason != DLL_PROCESS_ATTACH)
3         return;
4
5     void* startAddr = nullptr;
6     auto NtQueryInformationThread = (decltype(&::NtQueryInformationThread))
7         GetProcAddress(GetModuleHandleA("ntdll.dll"), "NtQueryInformationThread");
8
9     NtQueryInformationThread((HANDLE)-2, (THREADINFOCLASS)0x9,
10                             &startAddr, sizeof(startAddr), nullptr);
11
12     HMODULE hMods[128]; DWORD cbNeeded;

```

```

13 EnumProcessModules(GetCurrentProcess(), hMods, sizeof(hMods), &cbNeeded);
14
15 for (unsigned int i = 0; i < cbNeeded / sizeof(HMODULE); i++) {
16     MODULEINFO modInfo;
17     GetModuleInformation(GetCurrentProcess(), hMods[i], &modInfo, sizeof(modInfo)
18 );
19     if ((uintptr_t)startAddr >= (uintptr_t)modInfo.lpBaseOfDll &&
20         (uintptr_t)startAddr < (uintptr_t)modInfo.lpBaseOfDll + modInfo.
21         SizeOfImage)
22         return; // Thread is safe
23 }
24
25 printf("[!]_Suspicious_thread_start_address_detected\n");
26 }

```

Listing 4.1: Simplified TLS-based detection

Why TLS Callbacks Matter in Cheat Detection

TLS callbacks offer anti-cheats a method to intercept thread initialization in a way that doesn't rely on fragile hooks or active polling. Since callbacks run before the thread's entry point is reached, they allow anti-cheats to inspect the thread's context while it is still in a trustworthy state.

This makes them particularly effective against shellcode or Manual Mapping payloads that rely on stealthy execution.

3 Evasion and Limitations

Despite the effectiveness of Thread Local Storage (TLS) callbacks in detecting unauthorized module loads, advanced cheat developers have engineered multiple strategies to circumvent this mechanism. These techniques are specifically designed to minimize the visibility of injected code and blend execution seamlessly into the target process's control flow, reducing the likelihood of detection by anti-cheat systems that monitor TLS activity.

Bypassing TLS Callbacks

Several evasion strategies have emerged to avoid triggering TLS callbacks:

- **Overwriting or Removing the TLS Directory:** By modifying the PE headers of the injected module and either overwriting or zeroing out the TLS directory, cheats can effectively suppress the registration of TLS callbacks. This prevents the operating system from executing any associated initialization routines, thereby avoiding detection vectors tied to those callbacks.

- **Thread Hijacking:** Rather than spawning new threads—which are often monitored and can trigger TLS execution—cheats can hijack existing in-game threads. This involves suspending a legitimate thread, altering its instruction pointer (via `SetThreadContext`), and redirecting execution to the injected payload. Since the thread is already part of the game’s thread pool, it is less likely to be flagged by TLS-based or behavioral heuristics.
- **Trusted Thread Creation via Legitimate Modules:** Cheats may delegate thread creation to trusted game modules such as `tier0.dll` or `engine.dll`. By invoking internal thread management routines or mimicking the structure of native game threads, the injected logic can operate under the guise of legitimate execution flows. This approach takes advantage of the game’s own thread spawning mechanisms to obscure the presence of foreign code.
- **Return Address Spoofing:** When calling internal functions or executing injected code, return address spoofing is used to fabricate a legitimate call stack. This involves placing a plausible return address—typically pointing to a known game function or system API—on the stack before transferring control to the payload. Anti-cheat systems that perform stack walking or trace-based validation are thus deceived into interpreting the call as genuine.

ROP-Based Start Address Evasion

One particularly effective method to evade VAC’s `StartAddress` integrity checks during thread creation is to use a basic Return-Oriented Programming (ROP) technique. Rather than pointing the thread’s entry point directly to the injected code—which might raise suspicion—this approach utilizes a small instruction sequence (gadget) already present within a trusted game module.

A common gadget used for this purpose is the `‘jmp rcx’` instruction (`‘FF E1’` in hexadecimal), which can be found in several CS2 modules, including `client.dll`, `tier0.dll`, or even the main binary `cs2.exe`. Since these modules are part of the legitimate game distribution, using their code pages helps blend the injected thread into normal execution flows.

The technique works as follows:

1. Search for the opcode sequence `‘FF E1’` (representing `jmp rcx`) in any loaded game module (e.g., `cs2.exe`).
2. Pass the address of this gadget as the `lpStartAddress` parameter in a call to `CreateThread`.
3. Set `lpParameter` to point to your actual shellcode or mapped DLL entry point.

When the thread is created, it will begin execution at the gadget address. The `‘jmp rcx’` instruction will immediately redirect control to the payload stored in `rcx`, which

was preloaded with `lpParameter`. This technique not only circumvents direct jumps into untrusted memory regions but also preserves plausible execution flow through trusted module code.

This method is a minimalistic form of ROP chaining and is particularly valuable when combined with return address spoofing and thread mimicry. It allows the injected code to appear as if it originated from within CS2 itself, thereby evading thread-based and call stack integrity checks performed by VAC.

Limitations and Detection Risks

While these techniques provide a higher degree of stealth, they are not without limitations. For instance, removing the TLS directory may lead to compatibility issues with standard PE loaders or trigger suspicion during integrity verification. Thread hijacking introduces potential instability if the hijacked thread is performing time-sensitive operations. Similarly, return address spoofing must be carefully calibrated to avoid corrupting stack traces, especially when exceptions are thrown or debugging features are active.

Moreover, modern anti-cheat engines increasingly incorporate behavioral heuristics, kernel-level monitoring, and anomaly detection through machine learning, making even sophisticated evasion techniques susceptible to eventual detection. Thus, while these methods significantly reduce the likelihood of TLS-based detection, they must be implemented with precision and constantly adapted to evolving countermeasures.

4 VAC-Specific Capabilities

While the exact workings of VAC are undisclosed, community research shows that VAC inspects memory regions, thread start addresses, and stack traces. Suspicious patterns, such as start addresses in anonymous memory or shellcode trampolines, may be uploaded to Valve's servers for further analysis. Combined with delayed banning, this makes VAC harder to reverse-engineer.

5 Summary

Manual Mapping remains the most effective injection method in modern cheat development. Its stealth, control, and flexibility give it an edge over traditional techniques. However, VAC's increasing sophistication—particularly with TLS-based inspection—means that even Manual Mapping requires careful implementation. Avoiding detection today demands not just technical skill, but an intimate understanding of the Windows internals and anti-cheat heuristics.

5. INTERNAL GAME STRUCTURE: LOGIC BEHIND COUNTER-STRIKE 2

1 Understanding Entity Lists in Video Games

In modern game engines—particularly those designed for real-time, multiplayer environments—an entity list is a fundamental data structure responsible for managing all interactive objects within the game world. These entities include players, non-player characters (NPCs), projectiles, grenades, environmental props, and more.

At the core, each entity is an instance of a class containing essential data such as position, health, team ID, state flags, and animation or input components. The engine maintains and updates these entities each frame to simulate gameplay logic, render visuals, process collisions, and evaluate state-based behavior such as death, visibility, or interaction.

1.1 Common Structures for Entity Lists

Entity lists are implemented using various structures, depending on the performance needs and memory model of the engine:

- Array of Entity Objects – A flat, contiguous layout offering fast access, but limited flexibility.
- Array of Pointers – A static array referencing dynamically allocated entities. Used frequently in older engine architectures.
- `std::vector` – A dynamic container suitable for runtime entity creation and destruction.
- Linked List – Each node points to the next entity, providing memory flexibility at the cost of linear traversal time.
- HashMap / Tree – Enables fast lookup and categorization by entity ID or class type.

While early iterations of the Source engine (e.g., CS:GO) utilized a flat pointer array, the modern Source 2 engine in Counter-Strike 2 introduces a paged entity system. This structure maps entity handles to memory pages, offering greater scalability and safety across dynamically allocated objects.

2 Entity Organization in Counter-Strike 2

In Counter-Strike 2, every in-game object—be it a player, weapon, projectile, or environmental element—is represented as a dynamic entity. Internally, these entities are managed through unique handles, compact structures that encode both the entity index and a generation counter. This system enables safe referencing, preventing stale or invalid pointers when entities are destroyed and recreated during gameplay.

To resolve a handle into a valid pointer, the Source 2 engine employs a two-level paging system—conceptually similar to the virtual memory mechanisms used in modern operating systems—allowing efficient access to entity memory while maintaining safety and scalability.

When focusing on player-related data, two core classes emerge as central components:

- `CBasePlayerController` — Contains high-level player metadata such as the sanitized name, Steam ID, input state, and a handle to the corresponding pawn.
- `C_CSPlayerPawn` — Represents the physical player entity within the game world. It includes key gameplay attributes such as position, health, team ID, view angles, and animation state.

Both classes derive from the common superclass `C_BaseEntity`, which serves as the foundation for all entities in the engine. It exposes core functionality and network-relevant properties. Below are simplified, annotated representations of these structures:

1. Base Entity – `C_BaseEntity`

```
1 /*****
2  * C_BaseEntity:
3  * Base class for all entities in the Source 2 engine.
4  * Provides core fields used by all entity types.
5  *****/
6 class C_BaseEntity {
7 public:
8     CHandle<C_BaseEntity>    m_hEffectEntity;    // * Effect entity (e.g. glow,
9                                     particle)
10    CHandle<C_BaseEntity>    m_hOwnerEntity;      // * Owner entity (e.g. weapon
                                     holder)
    CHandle<C_BaseEntity>    m_hGroundEntity;      // * Entity this one is standing
                                     on (for physics)
```

```

11  CBodyComponent::Storage_t m_CBodyComponent;    // * Physics and collision
      representation
12
13  // * ... Additional base class members used engine-wide
14  };

```

Listing 5.1: C_BaseEntity – Core Class for All Entities

2. Player Controller – CBasePlayerController

```

1  /*****
2  * CBasePlayerController:
3  * Handles metadata, Steam ID, input state, and pawn reference.
4  * Represents the logical (non-physical) part of the player.
5  *****/
6  class CBasePlayerController : public C_BaseEntity {
7  public:
8      std::string      m_sSanitizedPlayerName; // * Cleaned player name (UI, logs)
9      uint64_t         m_steamID;             // * 64-bit unique Steam ID
10     InputState_t      m_inputState;          // * Current input (WASD, mouse,
      etc.)
11     CHandle<C_CSPlayerPawn> m_hPawn;          // * Handle to the in-game
      player pawn
12
13     // * ... Additional metadata like scores, latency, etc.
14 };

```

Listing 5.2: CBasePlayerController – Player Metadata and Input Handler

3. Player Pawn – C_CSPlayerPawn

```

1  /*****
2  * C_CSPlayerPawn:
3  * Represents the player's in-world presence (rendered and simulated).
4  * Stores spatial, animation, and gameplay-critical state.
5  *****/
6  class C_CSPlayerPawn : public C_BaseEntity {
7  public:
8      Vector3          m_vPosition;           // * Player's world-space position
9      uint32_t          m_iHealth;             // * Health (0 = dead)
10     uint32_t          m_iTeamNum;            // * Team ID (e.g., 2 = T, 3 = CT)
11     QAngle            m_angViewAngles;       // * Viewing direction (pitch/yaw/roll)
12     AnimationState_t   m_AnimState;          // * Animation and pose state
13
14     // * ... Additional attributes like velocity, inventory, flags, etc.
15 };

```

Listing 5.3: C_CSPlayerPawn – In-World Representation of the Player

This class hierarchy enables a clean separation of concerns between the logical identity and control of the player (controller) and their physical simulation within the world (pawn). This design also accommodates edge cases such as spectator modes, bot logic, and replay systems, where a controller may exist without an active pawn.

Moreover, the architecture reflects a variant of the Entity-Component-System (ECS) paradigm, promoting modularity and flexibility. This pattern is especially beneficial for external tools, including mods, analytics systems, and cheat frameworks.

2.1 Structuring Player Data in Practice: The Players Class

To organize the resolved player entities in a consistent and performant way, we define a high-level wrapper class called `Players`. This class aggregates all relevant information associated with a player in a single structure:

- The player’s in-game entity (`Pawn`), responsible for physical representation and simulation.
- The logical controller (`Controller`), which manages metadata, input, and team identity.
- The observed pawn (`ObserverPawn`), used for deathcam, spectator, or over-watch viewing.

This encapsulation allows us to treat each player as a self-contained unit, streamlining access across features such as ESP overlays, aimbots, spectator tools, or replay systems.

Entity Aggregation Class – `Players`

```
1 class Players
2 {
3 public:
4     C_PlayerPawn*      Pawn;
5     C_PlayerController* Controller;
6     C_PlayerPawn*      ObserverPawn;
7
8     Players()
9         : Pawn(nullptr), Controller(nullptr), ObserverPawn(nullptr) {}
10
11     Players(C_PlayerPawn* pawn, C_PlayerController* controller, C_PlayerPawn*
12             observerPawn)
13         : Pawn(pawn), Controller(controller), ObserverPawn(observerPawn) {}
```

Listing 5.4: Players – Unified Player Representation

3 Entity List Traversal

Having established the structural foundations of player entities, the next step involves enumerating these entities during runtime. This process—commonly referred to as entity list traversal—is fundamental for real-time game modification and data extraction.

3.1 Entity Handle Relationships

The linkage between `CBasePlayerController` and `C_CSPlayerPawn` is facilitated by the use of compact entity handles—typically implemented as `uint32_t` integers.

- `CBasePlayerController::m_hPawn` points to the controlled pawn.
- `C_CSPlayerPawn::m_Controller` points back to the controlling entity.

This bidirectional association enables fast, stable resolution of the player structure across different modules and states.

3.2 Locating and Identifying the Entity List in Memory

In the absence of debugging symbols or SDK access—common in protected or closed-source games—discovering the location and structure of the entity list becomes a crucial part of the reverse engineering process. This list is essential for iterating over active in-game entities such as players, bots, or interactive objects, and retrieving key data like position, health, and team.

The process is typically performed using memory inspection tools such as Cheat Engine, and can later be optimized or automated once the structure is confirmed. A generalized and practical methodology for discovering the entity list dynamically is outlined below.

Dynamic Discovery Workflow

1. Scan for the Local Player's Position: Begin by searching for changing float values corresponding to the local player's coordinates (e.g., X, Y, Z) as you move within the game.
2. Filter Non-Dynamic or Cached Results: Disregard addresses that stop updating when the game is paused or the window loses focus. These are often visual proxies or cached rendering data.

3. **Analyze Accessing Instructions:** Use Cheat Engine's "Find out what accesses this address" feature to locate game instructions interacting with the position. Valid update loops will continue accessing these addresses even when the game is idle.
4. **Detect Multi-Entity Access Patterns:** Verify whether the same instruction reads positions of other players. If so, it likely belongs to a loop iterating through a centralized entity container.
5. **Derive Entity Base Addresses:** Subtract the known position offset (e.g., `m_vPosition`) from each accessed address to recover the base pointer to each entity structure.
6. **Identify the Entity List Container:** Search for contiguous memory blocks where these base addresses are stored. Look for uniform spacing between them (e.g., every 0x78 or 0x80 bytes), which often indicates a static array or a paged entity structure.
7. **Validate and Generalize:** Once confirmed, build a loop to iterate over the list using pointer arithmetic and known offsets. From here, you can begin resolving additional properties like health, team, and state flags.

Entity List Variants: Visual vs. Gameplay-Complete Structures

During memory inspection, it is common to encounter multiple memory regions that resemble entity lists. These structures may vary significantly in terms of the data they expose:

- **Render-Only Lists:** Often contain just positional or interpolation data used by the renderer. These lists update frequently but lack gameplay logic.
- **Gameplay-Relevant Lists:** Contain complete entity structures with health, team, ammo, armor, and state flags—used by core game logic for decision-making and rule enforcement.

While render-only lists can be identified relatively quickly—typically within minutes—the discovery of fully populated gameplay-relevant structures often requires extended investigation. The process may involve correlating memory offsets, validating run-time behavior, or leveraging function-level reverse engineering in disassembly tools.

Advanced reverse engineers often turn to static analysis using tools such as IDA Pro, Ghidra, or Binary Ninja. These enable the inspection of vtables, RTTI metadata, or instruction-level access patterns that point to the actual entity list access logic inside the game binary.

This multi-path approach is extensively documented and discussed by reverse engineering communities. A particularly useful case study can be found in a dedicated

thread on the UnknownCheats forum¹, where users share strategies for locating and validating entity lists across multiple game engines.

In conclusion, identifying the correct entity list—whether through dynamic analysis or disassembly-guided pattern matching—is a foundational skill for external tooling. It unlocks the ability to systematically access and manipulate in-game objects with high confidence and reliability, forming the basis for any higher-level system such as ESP, aimbots, or replay parsers.

3.3 Reverse Engineering the Engine’s Entity Resolution Logic

Through static analysis of the CS2 binary, one can uncover the internal logic used to resolve handles. The following IDA signature corresponds to the function responsible for locating entities within the paged entity list:

Known Byte Pattern

```
1 81 FA ?? ?? ?? ?? 77 36 8B C2 C1 F8 09 83 F8 3F 77 2C
2 48 98 48 8B 4C C1 ?? 48 85 C9 74 20 8B C2 25 ?? ?? ?? ??
3 48 6B C0 78 48 03 C8 74 10 8B 41 10 25 ?? ?? ?? ??
4 3B C2 75 04 48 8B 01 C3
```

Decompiled Logic (Simplified)

```
1 while (true)
2 {
3     if ((v7 & 0x7FFF) <= 0x7FFE && (v7 >> 9) <= 63)
4     {
5         uint64_t page = *(uint64_t*)(entityList + 8 * (v7 >> 9) + 0x10);
6         if (page)
7         {
8             uint64_t entry = page + 120 * (v7 & 0x1FF);
9             if (entry)
10            {
11                if ((* (uint32_t*)(entry + 0x10) & 0x7FFF) == v7)
12                    break;
13            }
14        }
15    }
16    if (--v7 < 0)
17        break;
18 }
```

¹<https://www.unknowncheats.me/forum/general-programming-and-reversing/564058-entity-list.html>

```
19 *(uint32_t*)(entityList + 0x20F0) = v7;
```

This logic demonstrates how the engine navigates a paged entity list using high and low bits of the handle as page and slot indices, respectively. A verification step ensures that the slot matches the original handle, maintaining integrity.

3.3.1 Alternative Method: Hooking OnAddEntity and OnRemoveEntity

While iterating over the full entity list is a viable approach—as demonstrated in the previous section—it has limitations, especially in terms of safety and performance. A naive traversal risks accessing null or invalid entities, particularly in scenarios where:

- A player has recently disconnected.
- An entity was destroyed or reassigned (e.g., team switch or respawn).
- The entity memory is not yet initialized by the engine’s internal loop.

To mitigate these risks, we employ a more reliable and event-driven technique: hooking the engine’s entity lifecycle callbacks. Specifically, we intercept two key functions—OnAddEntity and OnRemoveEntity—which the engine internally invokes whenever an entity is added to or removed from the active game world.

By intercepting these functions, we maintain a synchronized internal representation of the entity list, without having to traverse memory manually every frame.

Hooked Callbacks for Entity Lifecycle

```
1 void HooksManager::OnAddEntity::hOnAddEntity(__int64 CGameEntitySystem, void*
   entityPointer, int entityHandle)
2 {
3     oOnAddEntity(CGameEntitySystem, entityPointer, entityHandle);
4
5     std::string schemaName = iGameEntitySystem->GetSchemaName(entityPointer);
6
7     if (schemaName == "C_CSPlayerPawnBase")
8     {
9         iGameEntitySystem->PawnMap[entityHandle] = (C_PlayerPawn*)entityPointer;
10    }
11
12    if (schemaName == "c_cs_observer_for_precache")
13    {
14        iGameEntitySystem->ObserverMap[entityHandle] = (C_PlayerPawn*)
           entityPointer;
15    }
16
```

```

17  if (schemaName == "CBasePlayerController")
18  {
19      iGameEntitySystem->ControllerMap[entityHandle] = (C_PlayerController*)
        entityPointer;
20  }
21  }

```

Listing 5.5: Hook: OnAddEntity

Lifecycle-Driven Synchronization

With this approach, the cheat framework reacts to entity creation and destruction in real time, mirroring the game's own internal management. This results in several key benefits:

- Performance: No need to scan memory every frame.
- Accuracy: No risk of accessing invalid or outdated entity pointers.
- Extensibility: Enables features like lifecycle analytics or event logging.

Initial Synchronization Step

It is worth noting that this hook-based method alone does not populate the entity list retroactively. Therefore, upon joining a game that has already started, we must perform a one-time full traversal of the entity list to capture entities that were added prior to our hooks being installed.

Once this initial population step is completed, entity state is fully managed via the OnAddEntity and OnRemoveEntity hooks—ensuring consistency, minimal overhead, and maximal safety.

6. CHEAT FEATURES IMPLEMENTED IN OUR CHEAT FOR CS2

This chapter provides a comprehensive analysis of the cheat features we have successfully implemented within our cheat. We will explore their objectives, the underlying logic governing their operation, and the technical implementation that enables them to function correctly. Each section will present code snippets that illustrate key parts of our implementation, allowing for a deeper understanding of how we interfaced with the CS2 engine. Furthermore, we will outline the specific functions we have hooked, demonstrating how we gain control over critical game processes to manipulate rendering, aiming, and other essential aspects of gameplay.

Functionality	Description
Chams	Modifies the rendering pipeline to apply custom materials to in-game models, enhancing visibility.
ESP (Extra Sensory Perception)	Extracts and displays real-time enemy information, providing a tactical advantage.
Aimbot	Automates aiming mechanisms to improve accuracy and precision.
Anti-Aim	Manipulates player model orientation to mislead enemy aimbots and disrupt targeting.

Table 6.1: Core Functionalities Overview

There exist multiple approaches to implementing each of these cheat features, ranging from simple external overlays to deep internal hooks that modify game memory and execution flow. Given the time constraints of this work and the limited publicly available documentation on cheat development, we have opted for an approach that provides the best balance between effectiveness and stealth.

The lack of official documentation and the inherently secretive nature of cheat development meant that much of our implementation had to be based on reverse engineering and extensive experimentation. Instead of relying on external overlays, which are more susceptible to detection and performance limitations, or resorting to aggressive memory manipulation, which significantly increases the risk of being flagged by anti-cheat systems, we chose to integrate directly with the game's internal systems.

We begin by exploring Chams, one of the most visually impactful modifications. This technique alters the appearance of in-game models by replacing their default materials with custom ones, effectively enhancing player visibility. Unlike traditional ESP methods, which rely on external overlays that merely draw information on the player's screen, Chams modify the rendering process within the engine itself. This allows for an entirely integrated solution that operates without any performance impact, making it one of the most effective and undetectable visual enhancements available.

In the following sections, we will dissect the logic behind our Chams implementation, detailing how we intercept key rendering functions, modify material properties, and dynamically apply custom textures to selected in-game entities. Each aspect of the implementation will be supported by relevant code snippets, providing an in-depth look at how we have programmed this feature.

1 Chams: Enhancing Visibility Through Engine-Based Rendering

Chams (chameleon skins) is a rendering manipulation technique that enables the replacement of standard in-game materials with custom-designed ones, significantly enhancing model visibility and allowing players to track enemy movements with greater ease regardless of lighting conditions, distance, or obstacles that would typically obscure their position.

Unlike external overlays, which function independently from the game and merely display additional information on the screen, Chams operate directly within the game engine, modifying the rendering logic at its core to ensure seamless integration and optimal performance without introducing additional processing overhead.

By leveraging hooking techniques within CS2's material system, it becomes possible to dynamically inject new textures and visual effects that are not present in the base game, allowing for a level of customization that surpasses conventional graphical modifications while maintaining the efficiency and fluidity of the rendering pipeline.

To achieve this level of modification, we implemented a multi-step process that ensures full integration with the game's rendering system. This process consists of the following key components:

- Extracting Default Materials from the Game Engine – Understanding and retrieving the default materials used by the game.
- Generating Custom Materials – Creating and defining new textures and visual effects that will replace the existing materials.

- Hooking CreateMaterial and Injecting Custom Materials – Intercepting the game’s material creation functions to replace default materials with our customized versions dynamically.
- Hooking OnDrawModelExecute and Overriding Materials – Modifying the rendering function responsible for drawing models to enforce the application of custom materials in real-time.

In the following sections, we will provide a detailed explanation of each of these steps, outlining how they contribute to the seamless integration of Chams into the CS2 rendering pipeline. We will delve into the technical specifics of how default materials are extracted, how custom materials are generated, and how we hook into the rendering functions to ensure that these modifications persist across all game instances.

1.1 Extracting Default Materials from the Game Engine

To fully understand how materials are structured and applied in CS2, we conducted extensive debugging of the rendering engine. By hooking into the OnDraw function, we intercepted the material creation process and successfully extracted the data of a default in-game material.

The following is an example of a material retrieved directly from the CS2 engine:

```

1 <!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}
2   format:generic:version{7412167c-06e9-4698-aff2-e63eb59037e7} -->
3   {
4     shader = "csgo_effects.vfx"
5     g_tColor = resource:"materials/dev/primary_white_color_tga_21186c76.vtex"
6     g_tNormal = resource:"materials/default/default_normal_tga_7652cb.vtex"
7     g_flColorBoost = 30
8     g_flOpacityScale = 245.55
9     g_vColorTint = [7.0, 7.0, 7.0, 0.37522]
10  }
```

Listing 6.1: Extracted Default Material from CS2

The extracted material follows the KV3 format, a modernized data structure introduced in Source 2 to replace the older KeyValues system used in Source 1.

KV3 allows for efficient serialization of structured data, supporting hierarchical organization and multiple data types, making it ideal for defining game assets such as materials, models, and particle effects. The Source 2 engine interprets these material definitions at runtime, applying the specified shader, texture maps, and rendering properties dynamically. By understanding this format, we gain full control over how materials are created and manipulated in CS2, enabling us to inject entirely new visual elements into the game without modifying pre-existing assets.

Further exploration of this format, along with an in-depth analysis of its structure and applications, was possible by studying the official Source 2 Developer Wiki, which provides extensive documentation on how Source 2 processes materials, shaders, and other graphical assets. This resource offered crucial insights into how KV3 interacts with the rendering engine, particularly in the way materials are compiled and utilized in real time.

By modifying this format, we successfully generated a variety of new materials, each possessing unique properties suited for different visual effects, allowing us to experiment with different rendering techniques and further refine our implementation of Chams in CS2.

1.2 Generating Custom Materials

After extracting and analyzing a default material from the game, we proceeded to experiment with different configurations to generate a collection of materials optimized for various Chams effects.

The goal of this process was to develop materials capable of altering in-game visibility while maintaining the integrity of CS2's rendering engine. By modifying specific attributes such as `g_flFresnelExponent` for light reflection, `g_flOpacityScale` for transparency, and the blending mode flags, we achieved a variety of effects including glow effects, metallic reflections, and enhanced visibility through obstacles.

```
1 std::vector <const char *> MaterialInfo = {  
2     szVMatBufferWhiteVisible,    szVMatBufferWhiteInvisible,  
3     szVMatBufferIlluminateVisible, szVMatBufferIlluminateInvisible,  
4     szVMatBufferIlatex,         szVMatBufferIlatexInvisible,  
5     szVMatBufferMetalic,        szVMatBufferMetalicInvisible,  
6     szVMatBufferGlowVisible,    szVMatBufferGlowInvisible  
7 };
```

Listing 6.2: List of Custom Materials

Each of these materials was assigned a name that reflects its primary visual characteristic to facilitate its integration into our cheat system. By storing these materials in a hashmap, we enable efficient retrieval and reuse whenever new materials need to be applied.

Since materials in CS2 are defined as structured strings that determine their rendering properties, we ensure that our injected materials adhere to this format. This allows us to seamlessly integrate custom materials into the game's rendering pipeline by hooking into the `CreateMaterial` function and dynamically replacing standard materials with our own.

One of the most visually distinctive materials we developed was the Glow Illuminated effect. This material was designed to create a glowing outline around player models

by leveraging CS2's built-in material masks and fine-tuning fresnel-based lighting parameters. The final definition of this material is shown below:

```
1 static static constexpr char szVMatBufferGlow2Visible[] =
2 R(<!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}format:generic:
   version{7412167c-06e9-4698-aff2-e63eb59037e7} -->
3 {
4     shader = "csgo_effects.vfx"
5     g_tColor = resource:"materials/dev/primary_white_color_tga_21186c76.vtex"
6     g_tNormal = resource:"materials/default/default_normal_tga_7652cb.vtex"
7     g_tMask1 = resource:"materials/default/default_mask_tga_344101f8.vtex"
8     g_tMask2 = resource:"materials/default/default_mask_tga_344101f8.vtex"
9     g_tMask3 = resource:"materials/default/default_mask_tga_344101f8.vtex"
10    g_flOpacityScale = 0.45
11    g_flFresnelExponent = 0.75
12    g_flFresnelFalloff = 1
13    g_flFresnelMax = 0.0
14    g_flFresnelMin = 1
15    F_ADDITIVE_BLEND = 1
16    F_BLEND_MODE = 1
17    F_TRANSLUCENT = 1
18    F_IGNOREZ = 0
19    F_DISABLE_Z_WRITE = 0
20    F_DISABLE_Z_BUFFERING = 0
21    F_RENDER_BACKFACES = 1
22    g_vColorTint = [1.0, 1.0, 1.0, 0.0]
23 });
```

Listing 6.3: Glow Illuminated Material Definition

This configuration takes advantage of Source 2's built-in rendering properties to produce a highly effective glow effect. The fresnel parameters control how light interacts with the surface, ensuring that the glow intensity changes dynamically based on the player's viewing angle. Additionally, the use of multiple texture masks helps the glow blend naturally into the surrounding environment, making it appear as a seamless part of the game's visual style rather than an artificial overlay.

By refining these materials through iterative testing and real-time observation, we successfully developed a collection of custom visual effects that seamlessly integrate into CS2's rendering engine. This process not only allowed us to enhance player visibility but also provided deeper insights into the flexibility of Source 2's material system. The ability to dynamically inject and apply new materials in real time offers significant advantages, as it enables rapid adjustments to the cheat's visual effects without requiring modifications to game files or precompiled assets.

Through the careful selection of material properties and the precise manipulation of CS2's rendering functions, we have created a system that allows for highly customizable visual enhancements. These materials serve as the foundation for our Chams implementation, which will be further detailed in the following sections.

1.3 Hooking CreateMaterial and Injecting Custom Materials

To dynamically create and register materials in CS2, we hooked into the CreateMaterialFunction, allowing us to manipulate how the game initializes and stores materials.

```
1 static int64_t (__fastcall *CreateMaterialFunction)(void *, void *, const char *, void *,  
    unsigned int, unsigned int);
```

Listing 6.4: Hooking CreateMaterial Function

With this function, we dynamically generated and injected new materials into the game:

```
1 CMaterial2 *CreateCustomMaterial(const char *materialName, const char *  
    materialData) {  
2     CKeyValue3 *pKeyValues = CreateMaterialResource();  
3     if (!pKeyValues) return nullptr;  
4  
5     LoadKeyValues(pKeyValues, nullptr, materialData, nullptr, nullptr, nullptr, nullptr,  
        nullptr, "");  
6     CMaterial2 *customMaterial = nullptr;  
7     CreateMaterialFunction(nullptr, &customMaterial, materialName, pKeyValues, 0, 1);  
8     return customMaterial;  
9 }
```

Listing 6.5: Creating a Custom Material in Memory

1.4 Loading KV3 Materials into the Game

After defining the custom materials, they must be properly loaded into the game engine so that they can be recognized and applied during rendering. This process involves parsing the KV3 data and ensuring that the Source 2 engine interprets it correctly. Without this step, the materials would remain as raw data in memory and would not be accessible for rendering operations. The following function is responsible for handling this process:

```
1 bool Visual::Chams::LoadKV3 (CUtilsBuff *buff, CKeyValue3 *CkeyVal)  
2 {  
3     KV3ID_t kv3ID = KV3ID_t ("generic", 0x41B818518343427E, 0  
        xB5F447C23C0CDF8C);
```

```

4  return LoadKeyValues (CkeyVal, nullptr, buff, &kv3ID, nullptr, nullptr, nullptr,
    nullptr, "");
5  }

```

Listing 6.6: Loading a KV3 Material into the Game

This function plays a critical role in ensuring that the Source 2 engine correctly processes and integrates the KV3 material data. The most notable aspect of this function is the initialization of the KV3ID_t object, which serves as a unique identifier for the KV3 data format. The string "generic" suggests that the format follows a standard KV3 structure. However, the most crucial part of this initialization lies in the two hexadecimal values: 0x41B818518343427E and 0xB5F447C23C0CDF8C.

These identifiers are likely used by the Source 2 engine as a reference signature for processing specific types of KV3 data. The first hexadecimal value, 0x41B818518343427E, may act as a format validation key that the engine checks to ensure that the data being loaded conforms to the expected KV3 structure. The second identifier, 0xB5F447C23C0CDF8C, could correspond to a particular category of KV3 files, distinguishing different types of resource files such as materials, models, or particle effects. This mechanism prevents malformed or incompatible KV3 data from being processed, ensuring that only correctly structured material definitions are registered in the engine.

Once the KV3 identifier is established, the function calls LoadKeyValues, which processes the KV3 data stored in memory. This function takes the parsed key-value pairs, ensures that the format aligns with the engine's expectations, and then integrates the material into the game's resource management system. The function parameters include a reference to the parsed data, a pointer to the KV3 identifier, and several nullptr arguments, which indicate that no additional optional parameters are required in this context.

By executing this function, the Source 2 engine successfully processes and registers the material, making it available for rendering in real-time. The use of predefined hexadecimal identifiers ensures that the KV3 data is correctly formatted and compatible with the rendering system. This approach enables the dynamic application of custom materials, allowing for real-time modifications without requiring precompiled assets or modifications to the game's existing files. The ability to inject materials dynamically provides a powerful method for implementing features such as Chams, enhanced weapon skins, and other visual modifications while maintaining full compatibility with the game engine.

1.5 Hooking OnDrawModelExecute and Overriding Materials

To dynamically apply Chams, we intercepted OnDrawModelExecute, the function responsible for rendering in-game models. By hooking into this function, we gained

control over the rendering process, allowing us to selectively modify materials applied to specific in-game entities. This method ensures that our custom materials are seamlessly integrated into the game's rendering pipeline without introducing additional rendering overhead or external overlays.

1.5.1 Filtering Target Entities

Since `OnDrawModelExecute` processes all rendered models, it is necessary to implement a filtering mechanism to ensure that only relevant entities—such as enemy players and weapons—are affected by our Chams system. Without this step, the modification would be applied indiscriminately to all objects in the game, including environmental elements and friendly players, which is neither efficient nor desirable.

To achieve this, we retrieve the schema name of each entity and compare it against known identifiers for player models and weapons. For player models, we check whether the entity corresponds to a `C_CSPlayerPawnBase` instance, which represents in-game characters.

```
1 auto schemaName = iGameEntitySystem->GetSchemaName(pEntity);
2 if (strcmp(schemaName.c_str(), "C_CSPlayerPawnBase") == 0) {
3     applyCustomMaterial(pMesh);
4 }
```

Listing 6.7: Identifying Player Models

Similarly, for weapons, we filter out entities corresponding to `C_PredictedViewModel`, ensuring that Chams are also applied to the player's held weapon model.

```
1 if (strcmp(schemaName.c_str(), "C_PredictedViewModel") == 0) {
2     applyCustomMaterial(pMesh, weaponMaterial);
3 }
```

Listing 6.8: Applying Chams to Weapons

By implementing this filtering mechanism, we ensure that Chams are applied exclusively to relevant gameplay elements, enhancing player visibility without interfering with other rendered objects.

1.5.2 Overriding Material Pointers

Once the relevant entities are identified, the next step is to override their assigned materials with our custom Chams textures. This is achieved by modifying the material pointer associated with the target model.

```
1 arrMeshDraw->CMaterial = *(CMaterial2 **)CustomMaterialMap.at(MaterialName);
```

Listing 6.9: Overriding Model Materials

This modification dynamically replaces the original material with one of our preloaded Chams materials stored within the CustomMaterialMap. By performing this override at runtime, we ensure that our changes take effect immediately, without requiring any modification to the game's original assets. This approach enables real-time visual adjustments and allows for the implementation of different Chams effects based on user preferences or situational requirements.

By leveraging this hooking technique, we achieve a stealthy and efficient modification of the rendering pipeline, integrating custom materials in a way that is indistinguishable from the game's native rendering behavior. This method not only enhances the effectiveness of Chams but also serves as a foundation for further graphical modifications within the cheat system.



Figure 6.1: Created Chams Materials

1.5.3 Visible vs Non-Visible Chams Logic

Once this logic is in place, we can take it a step further and assign different materials to the visible and non-visible parts of the enemy player model. This allows us to render, for example, the visible part of the player in one color (e.g., purple) and the non-visible part (e.g., behind walls) in another (e.g., yellow).

Since this rendering is only visible locally to the cheat user and not to other players in the match, it enables us to appear as a legitimate player with significantly better awareness and reflexes. By clearly seeing which parts of an enemy are exposed and which are not, we can ensure to only target the visible areas.

This reduces the risk of suspicious-looking shots (such as pre-firing through walls or headshots through solid cover) and helps us maintain a "legit" playstyle that blends in with skilled human behavior. In turn, this greatly decreases the chances of triggering manual bans during demo reviews or overwatch-like systems, where human moderators could flag unnatural behavior.



Figure 6.2: Enter Caption

2 Aimbot: Automated Aim to Enemy Players

An aimbot is one of the most fundamental and widely recognized cheat functionalities in competitive first-person shooters. It automates the aiming process by computing the precise trajectory required to target an enemy, thereby bypassing human limitations in reaction time and accuracy. Developing an aimbot for Counter-Strike 2 (CS2) necessitates a comprehensive understanding of the game's entity system, coordinate transformations, and trigonometric principles to ensure precise target alignment.

However, before implementing the aiming mechanism, it was crucial to first analyze how CS2 internally manages game entities. This understanding was essential since retrieving a player's position and calculating the required angles for the aimbot depend on accessing valid entity data. Without proper entity management, attempting to access a non-existent player could lead to dereferencing null pointers, resulting in critical errors or crashes.

2.1 Entity Management: Hooking the Game's Entity System

CS2 maintains an internal entity system that is responsible for handling all game objects, including players. Each entity is uniquely identified and assigned a memory address, allowing the game to track its state efficiently. Unlike other dynamic game

objects that are continuously updated, player entities in CS2 follow a more static lifecycle: they are loaded into memory once a player connects to a server and remain persistent unless a player disconnects or a new player joins the match. This means that, unlike projectiles or temporary objects, player entities are only modified under specific conditions, reducing unnecessary updates and improving performance.

To reliably access player data and maintain an up-to-date reference to all active players, we identified and hooked into two critical functions within the game's entity management system:

- **OnAddEntity** – Responsible for adding new entities to the game. When a player joins a match, the game dynamically allocates memory for a new entity, assigns it a unique identifier, and registers it in the entity system.
- **OnRemoveEntity** – Handles the cleanup and removal of entities when they leave the match. When a player disconnects or is kicked, the entity system marks their associated entity for deletion and releases the allocated memory.

By intercepting these functions, we effectively track player entities and store their references in a hashmap. This hashmap allows us to maintain real-time access to entity data while preventing invalid memory access. Since player entities are only modified when a new player joins or an existing player disconnects, the hashmap remains stable and does not require frequent updates, unlike entity lists that need to be re-queried every frame.

We implemented hooks for **OnAddEntity** and **OnRemoveEntity** to ensure our entity tracking remains consistent throughout a match. By leveraging hashmaps, we can efficiently store and retrieve player-related entities, reducing overhead and ensuring a seamless interaction with the game's entity system. Furthermore, this approach minimizes reliance on manually finding offsets, as entity references remain valid until explicitly removed by the engine.

```
1 void HooksManager::OnAddEntity::hOnAddEntity(__int64 CGameEntitySystem, void
   *entityPointer, int entityHandle) {
2   oOnAddEntity(CGameEntitySystem, entityPointer, entityHandle);
3
4   if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(), "
      C_CSPlayerPawnBase") == 0) {
5       iGameEntitySystem->PawnMap.insert(std::make_pair(entityHandle, (
          C_PlayerPawn *)entityPointer));
6   }
7   if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(), "
      CBasePlayerController") == 0) {
8       iGameEntitySystem->ControllerMap.insert(std::make_pair(entityHandle, (
          C_PlayerController *)entityPointer));
9   }
```

```
10 }
```

Listing 6.10: OnAddEntity Hook

```
1 void HooksManager::OnRemoveEntity::hOnRemoveEntity(__int64
    CGameEntitySystem, void *entityPointer, int entityHandle) {
2     oOnRemoveEntity(CGameEntitySystem, entityPointer, entityHandle);
3
4     if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(), "
        C_CSPlayerPawnBase") == 0) {
5         iGameEntitySystem->PawnMap.erase(entityHandle);
6     }
7     if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(), "
        CBasePlayerController") == 0) {
8         iGameEntitySystem->ControllerMap.erase(entityHandle);
9     }
10 }
```

Listing 6.11: OnRemoveEntity Hook

By correctly associating players with their respective controller and pawn structures, we ensure that all entity references remain valid.

2.1.1 Computing Aiming Angles

Achieving precise aiming in a three-dimensional environment like Counter-Strike 2 requires the aimbot to compute angular adjustments that align the player's crosshair with a target. Player orientation in CS2 is defined by three rotational angles:

- Pitch (θ): Controls vertical movement, determining how much the player looks up or down.
- Yaw (ϕ): Governs horizontal movement, rotating the player's view left or right.
- Roll (ψ): Represents tilt along the forward axis but remains fixed at zero in CS2.

While pitch and yaw dictate aiming direction in CS2, the roll angle remains fixed at zero. Unlike flight simulators or VR applications, where roll introduces an additional degree of freedom, CS2 enforces strict constraints that prevent camera tilting. Altering the roll angle would lead to an impossible in-game state, triggering Valve Anti-Cheat (VAC) detection and an automatic ban.

2.1.2 Deriving the Direction Vector

To accurately compute the aiming angles, the first step is to determine the enemy's position relative to the player's viewpoint. This process involves obtaining the player's position on the map, referred to as the foot position, and then retrieving

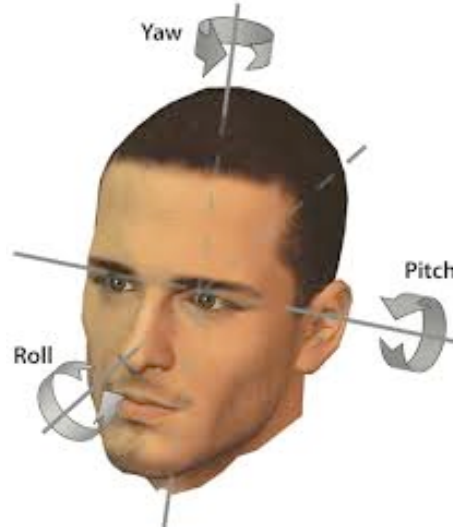


Figure 6.3: Representation of Pitch, Yaw, and Roll in a 3D Space.

the camera position, which represents the player's eye position. The eye position is the viewpoint from which the player perceives the game environment.

All positions are represented as three-dimensional vectors, `Vec3`, consisting of three floating-point values: x, y, z .

2.1.3 Computing the Player's Position

The player's final position in the game world is obtained by adding the base position of the player to the camera offset:

$$P_{\text{player}} = P_{\text{base}} + P_{\text{camera}}$$

Where:

- P_{player} is the final computed position of the player.
- P_{base} is the player's base position in the world.
- P_{camera} is the camera offset (view position).

Expressed in vector form:

$$\mathbf{P}_{\text{player}} = \begin{bmatrix} x_{\text{base}} \\ y_{\text{base}} \\ z_{\text{base}} \end{bmatrix} + \begin{bmatrix} x_{\text{camera}} \\ y_{\text{camera}} \\ z_{\text{camera}} \end{bmatrix}$$

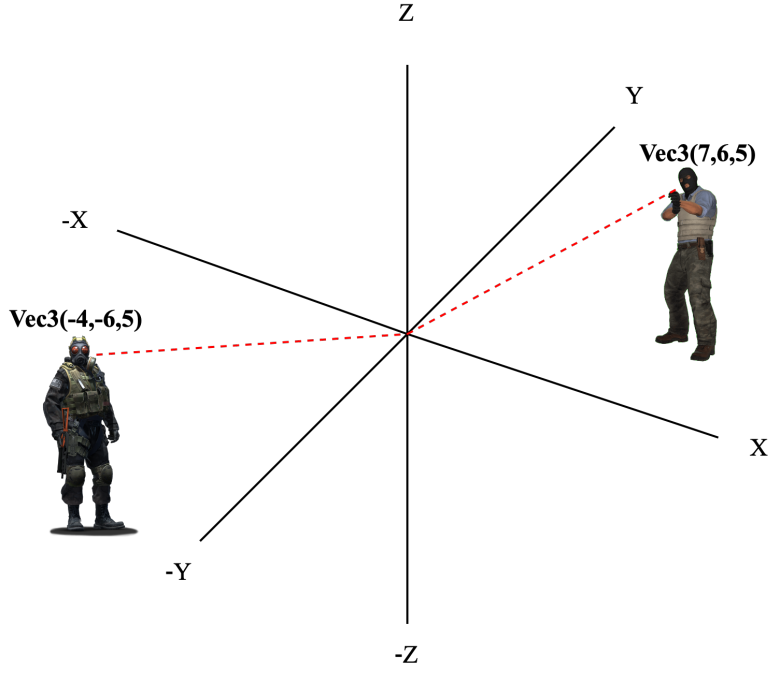


Figure 6.4: Representation of the player and enemy positions on the map.

2.1.4 Centering Coordinates on the Enemy

To center the coordinate system on the enemy, we compute the relative position of the enemy with respect to the player by subtracting the player's position from the enemy's position:

$$D = P_{\text{enemy}} - P_{\text{player}}$$

Expressed in vector form:

$$\mathbf{D} = \begin{bmatrix} x_{\text{enemy}} - x_{\text{player}} \\ y_{\text{enemy}} - y_{\text{player}} \\ z_{\text{enemy}} - z_{\text{player}} \end{bmatrix}$$

This transformation ensures that the player's position is effectively at the origin $(0, 0, 0)$, making it easier to compute aiming angles.

2.1.5 Local Coordinate System and Angle Calculation

As illustrated in Figure 6.6, all calculations take place in a local coordinate system where the player's eye position serves as the origin. The computed direction vector D , depicted by the blue line, serves as the foundation for determining the required pitch and yaw adjustments.

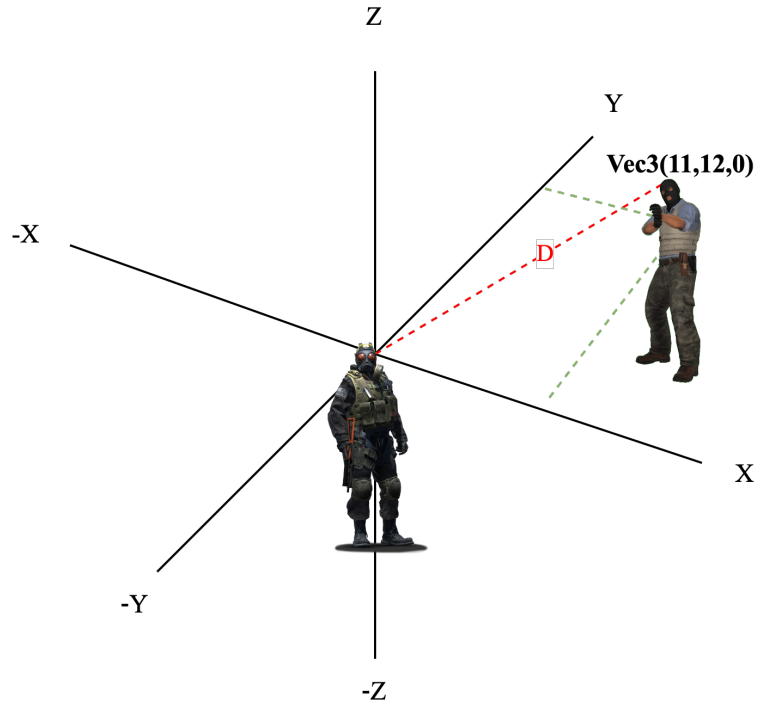


Figure 6.5: Computation of the displacement vector D from the player's eye to the target.

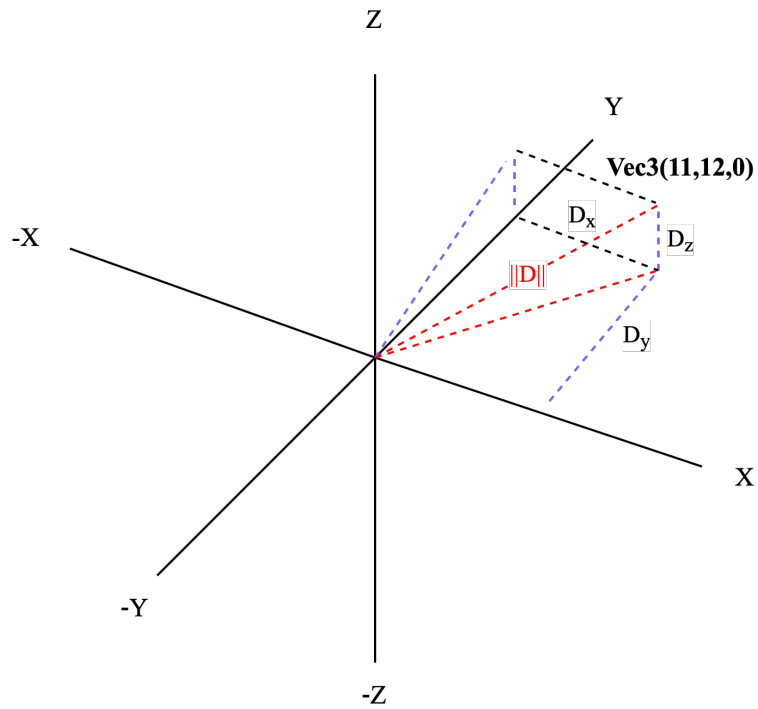


Figure 6.6: 3D representation of the aiming direction vector.

This displacement vector is essential for calculating the angles necessary to adjust the player's aim. The yaw and pitch angles can be derived using trigonometric functions, which will be explored in the next section.

2.1.6 Mathematical Computation of Aiming Angles

Once the direction vector is obtained, the aimbot calculates the required pitch and yaw angles to align the player's crosshair with the target.

Pitch Calculation (θ) The vertical adjustment necessary to match the enemy's head height is determined by:

$$\theta = \arcsin\left(\frac{D_z}{||D||}\right)$$

where:

$$||D|| = \sqrt{D_x^2 + D_y^2 + D_z^2}$$

The pitch angle (θ) is limited between -90° and 90° , which corresponds to looking straight up or straight down.

Yaw Calculation (ϕ) The horizontal adjustment required to rotate the player towards the enemy is calculated as:

$$\phi = \arctan 2(D_y, D_x)$$

The function $\arctan 2(D_y, D_x)$ is used instead of $\arctan(D_y/D_x)$ because it correctly determines the quadrant of the angle and prevents division errors when $D_x = 0$. The yaw angle (ϕ) spans from -180° to 180° , covering all possible directions.

Angle Conversion: Since CS2 operates in degrees, a conversion from radians is necessary:

$$\theta_{\text{degrees}} = \theta \times \frac{180}{\pi}, \quad \phi_{\text{degrees}} = \phi \times \frac{180}{\pi}$$

2.2 Overwriting Player View Angles

Once the aiming angles are computed, they must be applied to the player's in-game camera. This is achieved by hooking into the game's function responsible for setting view angles:

```
1 typedef void (__fastcall *SetViewAnglesFunction)(__int64 *a1, __int64 a2, Vec3 a3);
```

Listing 6.12: Overwriting Player View Angles

This function takes the following parameters:

- A pointer to the player's entity.
- An auxiliary parameter related to camera state.
- A vector `a3` containing the new pitch and yaw angles.

By injecting the computed angles into this function, the aimbot effectively reorients the player's view towards the target, ensuring near-instant alignment with minimal latency.

2.3 Smoothing for Stealth

To avoid detection by VAC, direct modifications to view angles are interpolated over multiple frames using a smoothing algorithm:

$$\theta_{\text{new}} = \theta_{\text{current}} + \frac{(\theta_{\text{target}} - \theta_{\text{current}})}{S}$$

$$\phi_{\text{new}} = \phi_{\text{current}} + \frac{(\phi_{\text{target}} - \phi_{\text{current}})}{S}$$

where S is the smoothing factor controlling transition speed. A larger smoothing value results in slower, human-like movements, significantly reducing the likelihood of detection.

By implementing smooth interpolation rather than instantaneously snapping to a target, the aimbot mimics natural aiming behaviors, thereby remaining undetected by VAC while maintaining effectiveness.

3 Bone Calculation: Extracting and Rendering Player Skeletons

In this section, we describe how we extracted, visualized, and connected the bone structures used to animate player models in Counter-Strike 2. These bones are essential for accurately representing player movement and orientation, and form the foundation for many ESP and aimbot features.

3.1 Extracting Bone Data from Player Entities

Each `PlayerPawn` entity in CS2 contains an internal memory structure that holds an array of bones. These bones represent all animation joints of the player model, from the pelvis and spine to the fingertips and eyes.

Through reverse engineering and memory analysis, we identified the layout of each bone entry in memory. Each bone follows the following structure:

```
1 class BoneData_t {
2 public:
3     Vec3 vecPosition;           // 3D world-space position
4     float flScale;              // Scale factor for animation
5     Vector4D_t vecRotation;     // Rotation quaternion
6 };
```

Listing 6.13: Bone Memory Structure

This structure allows us to read the world-space position of each bone, which is later projected to screen coordinates for overlay rendering.

To access these bones, we enumerated all relevant bone indices used by terrorist player models. After extensive experimentation, we constructed a detailed enumeration mapping each bone to its corresponding index in memory. The following snippet presents a partial view of the enumeration:

```
1 namespace terroristBones {
2     enum tBones : DWORD {
3         pelvis = 0,
4         spine_0 = 1,
5         spine_1 = 2,
6         spine_2 = 3,
7         spine_3 = 4,
8         neck_0 = 5,
9         head_0 = 6,
10        // ... continued through to:
11        leg_upper_r_twist1 = 103
12    };
```

13 }

Listing 6.14: Terrorist Bone Index Enumeration

This enumeration enables us to reference bones by name rather than raw index values, improving both readability and maintainability in our codebase.

3.2 Visualizing Bone Indices on the Player Model

After accessing the bone data, we developed a visualization tool to display each bone's index directly on the screen using our external overlay. This feature allows real-time debugging and inspection of the full bone structure.



Figure 6.7: Visualizing all bone indices rendered over the player model.

```

1 void RenderPlayerBones(C_PlayerPawn* player) {
2     if (player == GetLocalPlayer()) return;
3
4     for (int bone = terroristBones::pelvis; bone <= terroristBones::leg_upper_r_twist1;
5         ++bone) {
6         Vec3 worldPos = player->GetBone(bone)->vecPosition;
7         Vec2 screenPos;
8
9         if (!WorldToScreen(worldPos, screenPos)) continue;
10
11         ImU32 color = IM_COL32((bone * 15) % 255, (bone * 35) % 255, (bone * 55) %
12         255, 255);
13         DrawText(screenPos, std::to_string(bone).c_str(), color);
14     }
15 }

```

Listing 6.15: Displaying Bone Indices On-Screen

This system projects each bone's 3D position into 2D screen-space using the current game view matrix and renders its numeric index in a unique color. This was key for mapping bones during animation and ensuring accuracy in further rendering steps.

3.3 Rendering a Connected Skeleton

To enhance clarity and provide a meaningful representation of player posture, we implemented a simplified skeleton overlay. This system connects key bones with lines, forming a real-time stick-figure representation of the player's body.

```
1 void RenderSkeleton(C_PlayerPawn* player) {
2     if (!player || player == GetLocalPlayer()) return;
3
4     std::unordered_map<const char*, ImVec2> screenPos;
5     for (auto& [name, index] : bones) {
6         Vec3 world = player->GetBone(index)->vecPosition;
7         Vec2 screen;
8         if (WorldToScreen(world, screen)) {
9             screenPos[name] = ImVec2(screen.x, screen.y);
10        }
11    }
12
13    std::vector<std::pair<const char*, const char*>> links = {
14        {"head", "neck"}, {"neck", "torso"}, {"torso", "pelvis"},
15        {"torso", "left_clavicle"}, {"left_clavicle", "left_upper_arm"},
16        {"left_upper_arm", "left_lower_arm"}, {"left_lower_arm", "left_hand"},
17        ...
18        {"right_upper_leg", "right_lower_leg"}, {"right_lower_leg", "right_foot"}
19    };
20
21    for (auto& [a, b] : links) {
22        if (screenPos.count(a) && screenPos.count(b)) {
23            DrawLine(screenPos[a], screenPos[b], IM_COL32(255, 255, 255, 255), 2.0f);
24        }
25    }
26 }
```

Listing 6.16: Drawing a Simplified Player Skeleton

This skeleton representation provides a clean and intuitive view of player pose and animation, aiding not only in cheat visualization but also in development of features such as aim prediction, hitbox tracing, and movement recognition.



Figure 6.8: Simplified player skeleton rendered using bone connections.

BIBLIOGRAPHY

- Burnham, V. (2001). *Supercade: A visual history of the video game age*. MIT Press.
- Consalvo, M. (2007). *Cheating: Gaining advantage in video games*. MIT Press.
- Festinger, L. (1954). *A theory of social comparison processes* (Vol. 7). SAGE Publications. <https://doi.org/10.1177/001872675400700202>
- Kent, S. L. (2001). *The ultimate history of video games*. Three Rivers Press.
- McClelland, D. C. (1961). *The achieving society*. Princeton, NJ: Van Nostrand.
- Murdock, T. B., Anderman, E. M., & Hodge, S. A. (2001). Middle school teachers' classroom practices and adolescent cheating. *The Journal of Educational Psychology*, 93(2), 230–245. <https://doi.org/10.1037/0022-0663.93.2.230>
- Ramsay, M. (2012). *Gamers at work*. Apress.
- Smith, J. W. (2018). *The konami code: The history and impact of gaming's most famous cheat*. Game Studies Press.
- Wang, H. (2015). Reverse engineering memory structures using reclass.net. *Journal of Game Security*.
- Yan, J., & Randell, B. (2005). A systematic classification of cheating in online games. *Proceedings of Network and System Security*.
- Young, P. (2016). A brief history of online gaming and the rise of esports. *Digital Culture and Gaming*.